Co-4

* Data Analytics communication can refer to the process of using data to improve communication strategies or the skills needed to communicate data analysis results.

Key Aspects:-

1.) Know Your Audience
2.) Clarity & Simplicity.
3.) Data Storytelling.
4.) Effective use of visuals.
5.) Actionable Insights
6.) Iterative Feedback.
7.) Data Transparency.
8.) Use of Tools:- Like Tableau, Power BI etc...

Ex- → Retail Sales Dashboard.
→ Healthcare Analytics
→ Financial Forecasting Report.

Plotting

* Process of generating visual representations of data to enhance comprehension analysis & communication of information

Purpose of Plotting

* Data Exploration = Explores the characterstics and patterns.
* Data Analysis = Statistical analysis, hypothesis testing.
* Communication.
* Storytelling.

For Pairwise Data

* We use scatter, Bar, stem, stack and stairs plots.

for statistical Distribution

* We use Bar/Histogram, Box plot, Error Bar Plot, Violin Plot, Event Plot, Pie chart.

For Gridded Data

* We use imshow, pcolormesh, contour, contourf, barbs, quiver, streamplot.

For Irregularly gridded data

* We use tricontour, tricontourf, tripcolor, triplot.

3D & volumetric data

* We use scatter plot, plot_surface, plot_trisurf, voxels & wireframe plot.

Data types for plotting in PYTHON

→ int (Integer)

x = 5.

→ float

pi = 3.1415

→ str (String)

text = "Hello, world"

→ list

numbers = [1, 2, 3, 4, 5]

→ tuple

* similar to list but immutable i.e cannot be changed.

Coordinates = (10, 20)

→ dict

person = {'name': 'Alice', 'age': 30}

→ datetime

timestamp = datetime(2023, 11, 2)

→ Numpy arrays

Ex:- import numpy as np

data = np.array([1, 2, 3, 4])

→ bool

Ex:- is_valid = True

→ Set

Ex:- numbers = {1, 2, 3}

→ NaN

value = np.nan

Libraries

1) Matplotlib:- Used for creating static, animated, and interactive plots and arts.

2) Seaborn:- Provides a high-level interface for creating informative and attractive statistical graphics.

3) Pandas:- Has built in functions for creating basic plots using Matplotlib.
* Convenient for working with data frames.

4) Plotly:- Creates interactive, web-based visualizations.
* Supports wide range of chart types.

5) Altair:- This is a declarative statistical visualization.
* Simplifies the creation of charts using a simple & concise syntax.

6) Holoviews:- Creates interactive visualizations with a minimal amount of code.

7) Geopandas:- Works with geospatial data, creates maps and visualize geographic information.

8) Mayavi:- This is 3D visualization library that is particularly useful for scientific applications.

Line Plotting

* Line charts are used to represent the relation blw two data 'x' & 'y' on a different axis.

Ex:-

```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([1,2,3,4])
y = x**2
plt.plot(x,y)
plt.xlabel("x-axis");
plt.ylabel("y-axis");
plt.title("Any suitable title")
plt.show()
plt.figure() → Creates a new figure that can hold a new chart in it.
x1 = [2,4,6,8]
y1 = [3,5,7,9]
plt.plot(x1,y1,'--')
plt.show()
```

* We use matplotlib's plot for numerical data. (Line plot).
* Time Series Data :) matplotlib's plot or seaborn's lineplot

Scatter Plotting

* Data set is represented by dots.
* Observe relationships between variables and uses dots to represent the relationship blwn them.
* Used to represent relation among variables and how change in one affects the other.

Ex:- s = marker size
c = colors.

```
import matplotlib.pyplot as plt
import numpy as np.
x=np.array([5, 7, 8, 7, 2, 17, 2, 9, 4, 11, 12])
y=np.array([99, 86, 87, 88, 111, 86, 103, 87,
            94, 78, 77])
colors = np.array([0, 10, 20, 30, 40, 45, 50, 55, 60,
                   70, 80])
plt.scatter(x, y, c=colors, cmap='viridis',
            alpha=0.5)
plt.show()
```

* Value 0 = purple.
  Value 100 = yellow.
* alpha = adjust the transparency of the dots.
* We use edge color for outlining the markers.

Syntax → edgecolor='blue', linewidth=2)

Visualizing Errors

* Graphically represents the inaccuracies or deviations in model predictions.
* identify patterns in errors, detect biases, & evaluate model performance
* Confusion matrices, residual plots, and Roc curves visualize errors.

Confusion Matrix

* Used for classification
* Shows correct & incorrect predictions

Ex:-
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target, test_size=0.3, random_state=42)
model = RandomForestClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
disp.plot(cmap = plt.cm.Blues)
plt.show()
```

Residual Plot

* Regression.
* Checks model fit ; errors around zero.

Error Distribution Plot

* Regression
* Checks spread and bias of residuals.

Ex:-
```
import seaborn as sns
errors = y - ypred
sns.histplot(errors, kde=True)
plt.title("Error Distribution")
plt.xlabel('Error')
plt.ylabel('Frequency')
plt.show()
```

* kde = Kernel Density Estimate

Density Plot

* Represents the distribution & variation of data in two dimensions.
* Represents the distribution of continuous dataset.
* This is a variation of the histogram that uses kernel smoothing.
* The region of plot with a higher peak is the maximum data points residing between those values.

* Density plots can be visualized using pandas, seaborn.

Ex:-
```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
data = sns.load_dataset('car_crashes')
print(data.head(4))
data.speeding.plot.density(color='green')
plt.title('Density plot for Speeding')
plt.show()
```

* Univariate Analysis
* Analyze the distribution of a single variable, checking whether the data is symmetric, skewed or has multiple peaks.
* understands the differences blwn classes or clusters.
* Bivariate Analysis
* Used to visualize the joint distribution of the variables.
* Visualizes the correlation between two variables.

→ Outlier detection.
→ Model Evaluation

→ Probability Density Function (PDF)
* Kernel Density Estimation used to estimate the probability density function.
→ Data Exploration & Preprocessing.

Matplotlib

Ex:- Univariate Density plot.

```
import matplotlib.pyplot as plt
import numpy as np
data = np.random.randn(1000)
plt.hist(data, density=True, alpha=0.7,
         bins 30, color='blue', edgecolor
         xlabel = bl)
plt.title("U.D.P")
plt.xlabel('X-axis label')
plt.ylabel('Density')
plt.show()
```

Sea born

```
import seaborn as sns
import numpy as np
data = np.random.randn(1000)
sns.histplot(data, kde=True, color='green',
             bins=30)
plt.title('Univariate Density plot with
          Seaborn')
plt.xlabel('X-axis label')
plt.ylabel('Density')
plt.show()
```

Contour Plot

* 3D data is represented in 2D space.
* Used in scientific and engineering fields to visualize functions of two variables.
* Contour Plots show lines or contours that connect points with the same function value, creating a map of equal values.
* Same curve = Same Function value.

Ex:-

```
X = np.linspace(-5, 5, 100)
Y = np.linspace(-5, 5, 100)
X, Y = np.meshgrid(X, Y)
Z = np.sin(np.sqrt(X**2 + Y**2))
plt.contour(X, Y, Z, levels=20, cmap='viridis')
plt.colorbar(label='Function Value')
plt.title('Contour Plot')
plt.xlabel('X-axis label')
plt.ylabel('Y-axis label')
plt.show()
```

Binnings

* To find duplicate values, stopwords, different formatting we use binning.
* Process of data mining.
* Also known as discretization or Bucketing.

Working

Step 1:- Sort the dataset in ascending order.

Step 2:- Width of each bin using the formula: $w = (max - min)/N$.

N → Number of bins.

Ex:- 10, 15, 18, 20, 31, 34, 41, 46, 51, 53, 54.

min = 10 bins = 4

max = 54

$w = [54 - 10)/4 = 44/4 = 11$

Bin 1 => $[(min, (min + w - 1)] = [10, 20]$

Bin 2 => $[(min+w, (min+2w - 1)] = [21, 31]$

Bin 3 => $[(min+2w), (min+3w - 1)] = [32, 42]$

Bin 4 => $[(min+3w), (max)] = [43, 54]$